

Migration Map

Generated at: 7/25/2025, 8:12:29 AM

Files	Migration Plan	Suggested Tech	Database Suggestion	Note
config.js	1. Convert the config.php file to a config.js file in Node.js syntax. Update any necessary configurations to match the Node.js environment. 2. Create separate modules for handling user and payment functionalities in Node.js.	For handling database operations in Node.js, you can use libraries like Sequelize or TypeORM for ORM functionalities. These libraries will help in managing database connections, defining models, and executing queries.	You can migrate the existing MySQL database schema to a modern database solution like PostgreSQL or MongoDB. Using an ORM library like Sequelize or TypeORM will abstract the database operations and make it easier to switch between different database systems.	When converting from PHP to Node.js, pay attention to differences in syntax, asynchronous nature of Node.js, and handling of dependencies. Make sure to thoroughly test the migrated code to ensure functionality remains consistent.
PaymentService.php UserController.php User.php	1. Refactor PaymentService.php, UserController.php, and User.php into separate modules in Node.js. Use ES6 classes to represent each entity (User, PaymentService) and import them as needed in the main application file. Ensure proper dependency	For the Node.js equivalent technology, consider using Express.js for building the REST API, Sequelize ORM for database interactions, and Joi for input validation.	Migrate the database schema to a modern relational database like PostgreSQL or MySQL. Use Sequelize ORM to define models and handle database operations. Consider using migrations to manage schema changes.	When converting from PHP to Node.js, pay attention to differences in syntax, asynchronous nature of Node.js, and handling of dependencies. Make sure to handle error handling and logging appropriately in the Node.js application.

Files	Migration Plan	Suggested Tech	Database Suggestion	Note
	management between modules.			
schema.sql	1. Convert the schema.sql file to a Node.js module that exports the database schema as JSON objects. 2. Create separate Node.js modules for handling database operations for each table (e.g., users.js, payments.js) that use the exported schema objects. 3. Refactor any PHP code that interacts with the database to use the new Node.js modules.	Use Sequelize ORM for Node.js to handle database operations and migrations. Sequelize supports MySQL, which is compatible with the SQL schema provided.	Migrate the MySQL database to a modern database solution like PostgreSQL or MongoDB, depending on the specific requirements of the application. Use Sequelize ORM to handle database interactions in Node.js.	When converting from PHP to Node.js, pay attention to differences in syntax, asynchronous nature of Node.js, and handling of dependencies. Make sure to test thoroughly to ensure the migration is successful.